### 4.5 Algorithm of PrismSpace Multi-Agent System

The PrismSpace algorithm orchestrates specialized autonomous AI agents into an adaptive collaborative swarm: parsing complex user objectives, routing sub-tasks to cost- and latency-optimal LLM providers, dispatching specialized worker agents equipped with Model Context Protocol (MCP) tools, and enforcing rigorous preference alignment and safety gating. The algorithm operates across three cohesive phases:

- **Phase I: Offline Swarm Graph Representation & Routing Optimization** (Steps 1–3: Swarm graph topology, Graph Attention Network embedding, multi-task routing and dispatch loss).
- **Phase II: Single-Stage Preference Alignment** (Step 4: Unified SFT and Odds-Ratio preference tuning via ORPO without external reward models).
- **Phase III: Online Runtime DAG Orchestration & Swarm Execution** (Steps 5–7: Task subsection division, DAG construction, topological scheduling, ReAct tool execution, safety gating, and telemetry).

**System Inputs & Outputs:**

- **Inputs:** User objective prompt $\mathcal{T}_{\text{user}}$, curated training corpora $\mathcal{D}$ (intent, routing, and safety traces across RouterBench, LMSYS, WildGuard, CMU Agent benchmarks), preference pairs $\mathcal{D}_{\text{pref}} = \{(x, y_w, y_l)\}$, and global agent/tool registry $\mathcal{G}_{\text{global}}$.
- **Outputs:** Dispatched agent team $\hat{\mathbf{y}}_{\text{agents}}$, optimal LLM provider assignment $\hat{y}_{\text{provider}}$, execution DAG $\mathcal{G}_{\text{active}}$, verified streaming deliverables, and persistent session state in browser IndexedDB (`PrismDB`).

**Phase I: Offline Swarm Graph Representation & Intelligent Routing**

**Step 1: Swarm Graph Formulation ($\mathcal{G} = (V, E)$)**

1. *Node Definition ($V$):* Represent each specialized agent persona (Supervisor, Research, Coding, Validator, Planner) and each MCP tool capability (Browser, Filesystem, GitHub, Sandbox CLI) as a distinct vertex $v_i \in V$. Each node is initialized with a normalized capability feature vector $\mathbf{h}_i \in \mathbb{R}^d$.
2. *Collaborative Edge Topology ($E$):* Establish undirected communication edges $e_{ij} \in E$ between complementary agent-tool and agent-agent pairs whose functional correlation exceeds an empirical interaction threshold $\tau$, permitting direct message passing.

**Step 2: Attention-Based Message Passing (Graph Attention Network)** To dynamically model swarm affinity and contextual synergy, node embeddings are updated using multi-head graph attention:

$$\alpha_{ij}^k = \frac{\exp\Big(\text{LeakyReLU}\big((\mathbf{a}^k)^T[\mathbf{W}^k\mathbf{h}_i \,\|\, \mathbf{W}^k\mathbf{h}_j]\big)\Big)}{\sum_{m\in\mathcal{N}(i)} \exp\Big(\text{LeakyReLU}\big((\mathbf{a}^k)^T[\mathbf{W}^k\mathbf{h}_i \,\|\, \mathbf{W}^k\mathbf{h}_m]\big)\Big)}, \qquad \mathbf{h}_i' = \|_{k=1}^K \sigma\left(\sum_{j\in\mathcal{N}(i)} \alpha_{ij}^k \mathbf{W}^k\mathbf{h}_j\right) \tag{1}$$

*Notation:* $\mathbf{h}_i \in \mathbb{R}^d$ is the node feature vector; $\mathcal{N}(i)$ is the set of collaborative neighbors of node $v_i$; $\mathbf{W}^k \in \mathbb{R}^{d'\times d}$ is the head-specific linear transformation matrix; $\mathbf{a}^k \in \mathbb{R}^{2d'}$ is the parameterized attention mechanism vector for head $k \in \{1, \ldots, K\}$; $\|$ denotes vector concatenation; $\alpha_{ij}^k$ is the normalized attention coefficient reflecting how critical node $v_j$ is to node $v_i$; and $\mathbf{h}_i' \in \mathbb{R}^{K\cdot d'}$ is the updated context-aware node embedding.

**Step 3: Multi-Task Joint Routing and Agent Dispatch Objective** The pooled swarm representation $\mathbf{h}'$ and individual agent states $\mathbf{h}_i'$ are projected through dual prediction heads:

1. *Provider Routing Head:* Computes posterior probability distribution $\hat{\mathbf{y}}_{\text{route}} = \text{Softmax}(\mathbf{W}_{\text{route}}\mathbf{h}' + \mathbf{b}_{\text{route}})$ over available LLM providers (Groq for ultra-low latency, OpenAI GPT-4o for deep reasoning, Claude 3.5 Sonnet for code).
2. *Agent Dispatch Head:* Computes independent activation probabilities $\hat{y}_v = \text{Sigmoid}(\mathbf{w}_{\text{agent}}^T\mathbf{h}_v' + b_v)$ for each candidate agent persona $v \in V$.
3. *Joint Optimization Loss:* Parameters are trained end-to-end via AdamW using the multi-task objective:

$$\mathcal{L}_{\text{class}} = \mathcal{L}_{\text{CE}}(\mathbf{y}_{\text{route}}, \hat{\mathbf{y}}_{\text{route}}) + \lambda \sum_{v\in V} \mathcal{L}_{\text{BCE}}(y_v, \hat{y}_v) \tag{2}$$

where $\mathcal{L}_{\text{CE}}$ is categorical cross-entropy penalizing incorrect LLM provider routing against ground truth $\mathbf{y}_{\text{route}}$, $\mathcal{L}_{\text{BCE}}$ is binary cross-entropy penalizing over- or under-provisioning of agents against optimal ground truth $y_v \in \{0, 1\}$, and $\lambda$ is a balancing hyperparameter.

**Phase II: Single-Stage Preference Alignment (ORPO)**

**Step 4: Odds Ratio Preference Optimization** Rather than relying on unstable multi-stage Reinforcement Learning from Human Feedback (RLHF) requiring a separate reward model and PPO policy updates, PrismSpace tunes the agent models in a single training stage via Odds Ratio Preference Optimization (ORPO) paired with Low-Rank Adaptation (LoRA, $\Delta\mathbf{W} = \frac{\alpha}{r}\mathbf{BA}$):

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} + \beta \cdot \mathcal{L}_{\text{OR}}, \qquad \mathcal{L}_{\text{OR}} = -\log \sigma \left( \log \left[ \frac{\text{odds}(y_w \mid x)}{\text{odds}(y_l \mid x)} \right] \right) \tag{3}$$

*Notation:* $x$ is the input instruction prompt; $y_w$ is the winning/favored response; $y_l$ is the losing/disfavored response (e.g., hallucinated facts, ungrounded actions, safety policy breaches); $\mathcal{L}_{\text{SFT}} = -\frac{1}{|y_w|} \sum_{t=1}^{|y_w|} \log P(y_{w,t} \mid x, y_{w,<t})$ is the standard supervised causal language modeling loss maintaining fluency; $\text{odds}(y \mid x) = \frac{P(y|x)}{1-P(y|x)}$ calculates the length-normalized sequence odds; $\sigma(\cdot)$ is the sigmoid function; and $\beta$ is a weighting coefficient penalizing generation paths that favor rejected completions.

**Phase III: Task Subsection Division & Directed Graph Construction**

A central mechanism of the PrismSpace algorithm is decomposing an unstructured, high-level user objective $\mathcal{T}_{\text{user}}$ into an executable, dependency-aware workflow. This decomposition and graph synthesis operates as follows:

**1. What Vertices ($V_{\text{task}}$) Are Made Of:**

- When prompt $\mathcal{T}_{\text{user}}$ is ingested, the Supervisor/Planner agent decomposes the prompt into $N$ discrete, atomic task subsections: $V_{\text{task}} = \{v_1, v_2, \ldots, v_N\}$.
- Each vertex $v_i \in V_{\text{task}}$ represents an independent operational sub-task constructed from five concrete elements:

$$v_i = \big(\text{TaskID}_i,\ \text{Goal}_i,\ \mathcal{I}_i,\ \mathcal{O}_i,\ \text{Role}_i,\ \text{Provider}_i,\ \text{Tools}_i\big)$$

  where $\text{Goal}_i$ is the target instruction; $\mathcal{I}_i$ is the required input schema; $\mathcal{O}_i$ is the deliverable output artifact (e.g., CSV data, research summary, code patch); $\text{Role}_i \in \{\text{Planner}, \text{Researcher}, \text{Coder}, \text{Assistant}, \text{Reviewer}\}$ is the assigned agent persona; $\text{Provider}_i$ is the routed LLM backend; and $\text{Tools}_i$ is the set of granted MCP tools.
- This isolates concerns: each agent receives only its necessary inputs, eliminating context bloat and token waste.

**2. What Each Edge ($E_{\text{dep}}$) Does & How Vertices and Edges Connect:**

- *How They Connect:* An edge is a directed link $e = \langle v_i, v_j \rangle \in E_{\text{dep}}$ pointing from producer vertex $v_i$ to consumer vertex $v_j$, linking the output interface of $v_i$ directly to the input interface of $v_j$.
- *What Each Edge Does:* Each edge performs two operational functions:
  1. **Data Pipeline:** Automatically transmits the completed output artifact $\mathcal{O}_i$ from $v_i$ into the input buffer $\mathcal{I}_j$ of $v_j$.
  2. **Execution Gate:** Blocks task $v_j$ from starting until task $v_i$ completes its execution and delivers $\mathcal{O}_i$.

**3. Basis for Connecting Adjacent Nodes (Data Dependency Matching):**

- Adjacent nodes are connected **strictly on the basis of Data Dependency**: two nodes $v_i$ and $v_j$ are linked with directed edge $\langle v_i, v_j \rangle$ **if and only if** task $v_j$ requires the output artifact generated by task $v_i$:

$$\langle v_i, v_j \rangle \in E_{\text{dep}} \iff \mathcal{O}_i \cap \mathcal{I}_j \neq \emptyset$$

- *Concrete Connection Example:* For a request like *"Fetch quarterly financial metrics, scrape market news, synthesize executive insights, and format an email draft"*:
  - Nodes $v_1$ (Fetch CSV) and $v_2$ (Search News) have no input dependencies → they have no incoming edges.
  - Node $v_3$ (Synthesize Analysis) requires inputs from both $v_1$ and $v_2$ ($\mathcal{I}_3 = \{\mathcal{O}_1, \mathcal{O}_2\}$) → edges $\langle v_1, v_3 \rangle$ and $\langle v_2, v_3 \rangle$ connect them.
  - Node $v_4$ (Draft Email) requires $\mathcal{O}_3$ ($\mathcal{I}_4 = \{\mathcal{O}_3\}$) → edge $\langle v_3, v_4 \rangle$ connects them.
- *Parallel Concurrency for Independent Nodes:* Tasks with no connecting dependency edges (in-degree $= 0$, such as $v_1$ and $v_2$) execute simultaneously in parallel (concurrency limit $K \leq 3$). When $v_1$ and $v_2$ complete, their outgoing edges release $v_3$ for execution.

**Phase IV: Online Swarm Execution, Tool Interaction & Verification**

**Step 5: Autonomous ReAct Swarm Execution** For each scheduled task subsection $v_i \in \pi$:

1. Dispatched agent $v_i$ instantiates an autonomous Reasoning-Action (ReAct) cycle:

$$\text{Observation}_t \longrightarrow \text{Thought}_t \longrightarrow \text{Action}_t(\text{Tool}, \text{Params}) \longrightarrow \text{Observation}_{t+1}$$

2. The agent issues JSON-RPC calls across Model Context Protocol (MCP) server endpoints (e.g., executing Python scripts in a sandboxed CLI, writing files via Filesystem MCP, querying live APIs via Browser MCP).
3. Context is dynamically updated: $C_{t+1} = \text{Compress}(C_t \cup \Delta C_t \setminus D_t)$, evicting stale tool observations $D_t$ to prevent token window overflow.

**Step 6: Multi-Criteria Safety Gating and Anomaly Isolation** Before any generated deliverable is released to downstream tasks or the user interface, it must pass an automated multi-criteria safety evaluation barrier:

$$S = \sum_{m=1}^{M} w_m \, q_m \geq \tau_{\text{safety}} \tag{4}$$

where $q_m \in [0, 1]$ are individual verification scores across: (a) WildGuardMix adversarial prompt/harm checks, (b) IsolationForest tool parameter anomaly scores, (c) schema completeness against target specification $\mathcal{O}_i$, and (d) ORPO alignment safety margins; $w_m$ are normalized criterion weights ($\sum w_m = 1$); and $\tau_{\text{safety}}$ is the strict release threshold. Deliverables violating safety gating are redirected back to the Planner for corrective re-planning.

**Step 7: Deliverable Rendering and Reactive IndexedDB Persistence**

1. Verified partial results, reasoning traces, and artifacts are rendered live in the Next.js frontend via reactive local database subscriptions (`useLiveQuery()`).
2. The complete execution trace, including DAG state, agent conversation turns, tool call arguments, and final artifacts, is committed atomically to the browser's local IndexedDB instance via Dexie.js (`PrismDB`), guaranteeing zero data loss, offline accessibility, and total user privacy.

**Model Evaluation and Telemetry Diagnostics**

- *Routing and Safety Generalization:* Evaluated across holdout test splits ($\mathcal{D}_{\text{test}}$) using Macro-F1 (routing accuracy), ROC-AUC / PR-AUC (safety discrimination), and Expected Calibration Error (ECE $< 0.05$).
- *Swarm Telemetry HUD:* Real-time operational tracking of task execution latency $\hat{\ell}$ (ms), cumulative token costs $\hat{c}$ (USD), agent utilization rates, and cache hit ratios.

**Algorithm Summary & Mathematical Guarantee**

- **Deterministic Execution Order:** Decomposing user goals into task subsections $V_{\text{task}}$ connected by data dependency edges $E_{\text{dep}}$ guarantees that each task receives its required input artifacts before starting.
- **Optimal Resource Allocation:** GAT swarm attention and multi-task loss $\mathcal{L}_{\text{class}}$ ensure that only strictly necessary agents are instantiated and routed to cost- and latency-optimal LLM providers.
- **Single-Stage Alignment Stability:** ORPO loss $\mathcal{L}_{\text{ORPO}}$ enforces safety and human preference directly during parameter tuning without the instability, latency, or compute overhead of external reward models.